

macOS Reverse Proxy 3.1.7

In smaller organizations, it may be necessary to run more than one server on the same computer or run more than one server behind a single WAN i.p. address. When macOS Server was a thing, this guide helped run multiple server apps/appliances like FileMaker Server, Kerio Connect, Maxum Rumpus and Synology NAS on the same computer/network as macOS Server.*

When macOS Server stopped providing web services (in 10.14 Mojave) I still had some organizations using Reverse Proxy who needed to upgrade the OS for security and other reasons. So I have developed this guide to do Reverse Proxy using the native Apache web server built into every macOS from Mojave to macOS 27 Golden Gate.

***Not saying it is "Best Practices", nor that we recommend it; just saying you can do it.**

This version of the tutorial has been tested on macOS 15 Sequoia. For earlier versions refer to the chart below.

OS Version	Tutorial version
OS X 10.9-10.10 (OS X Server)	v.1.0.10
OS X 10.11 (OS X Server)	v.2.0.6
macOS/macOS Server 10.12 -13	v.2.0.6
macOS 10.14 - 10.15	v.3.1.7
macOS 11 - 27	v.3.1.7

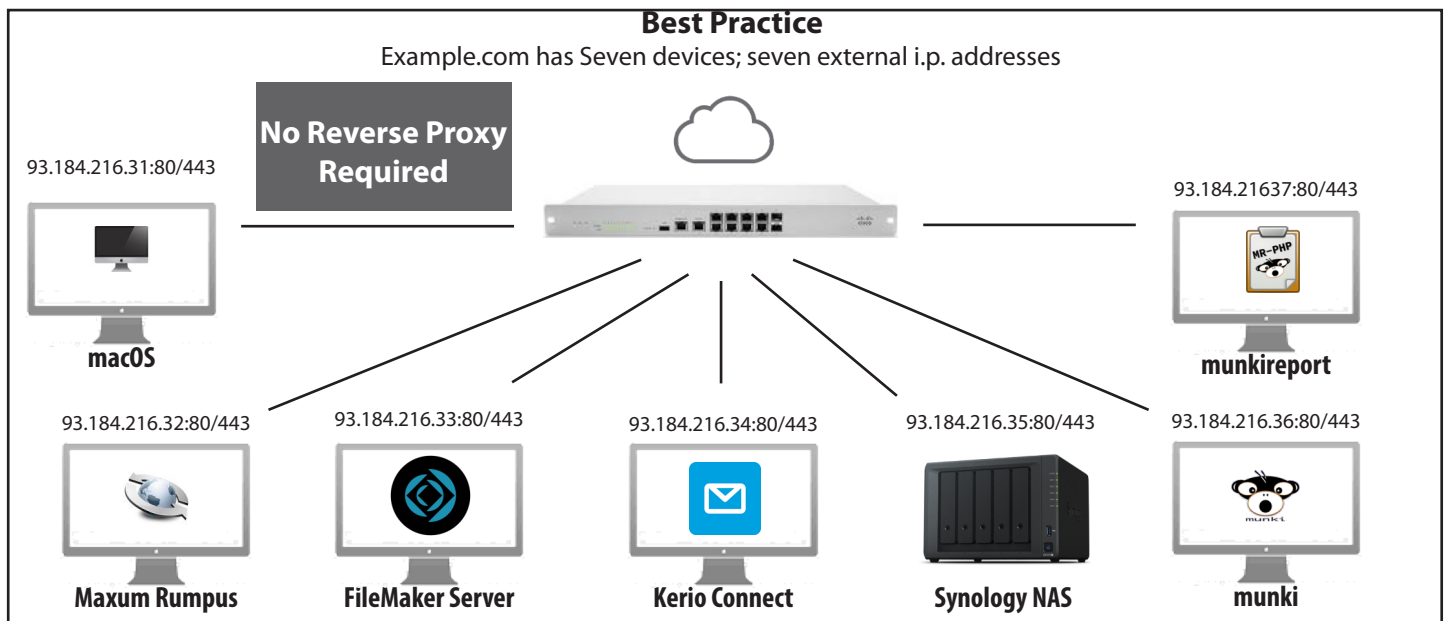


Fig. 1 DNS In Best Practice the enterprise level router uses four external i.p. addresses for four internal boxes. Each can have its own port 80 and port 443 traffic. Sweet!

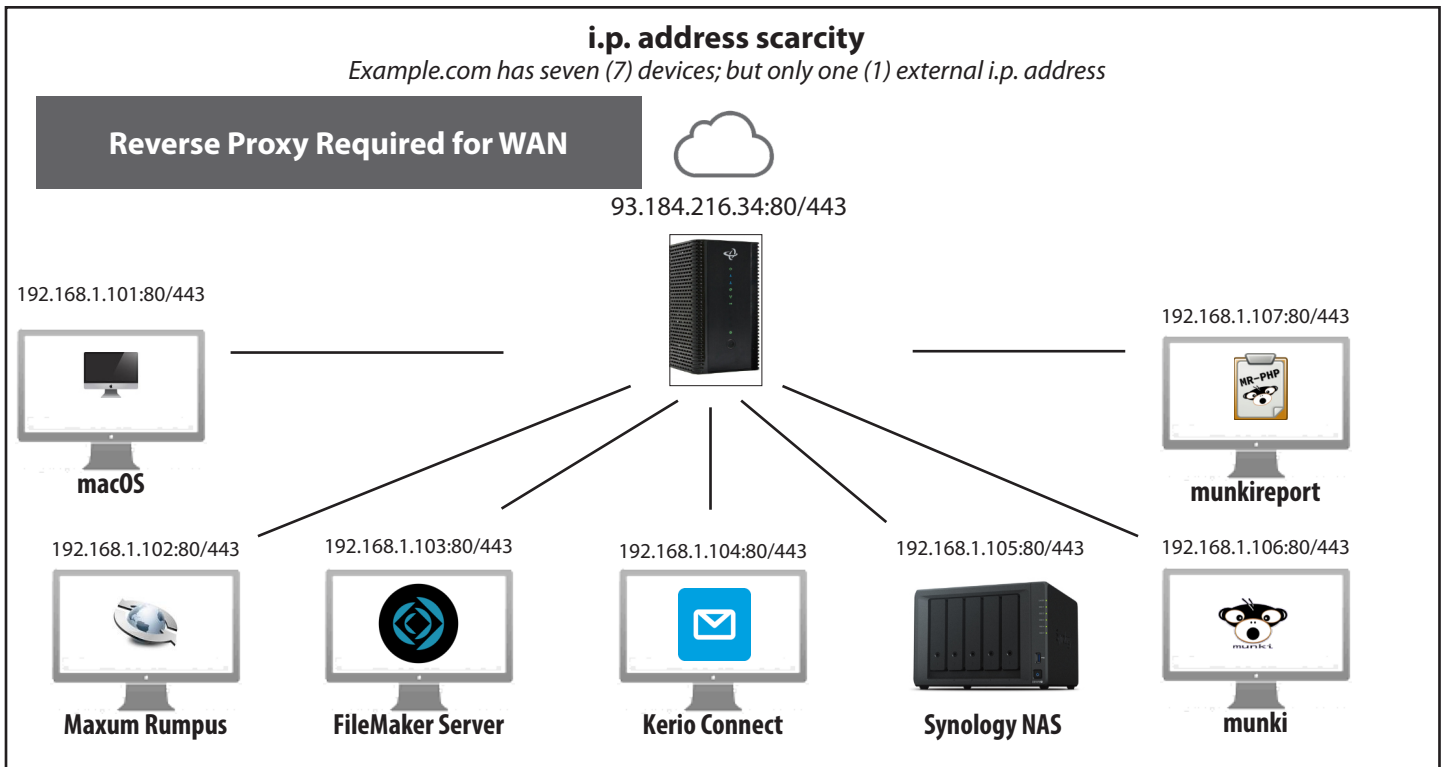


Fig. 2 There is only one external i.p. address. The router can only pass port 80 and 443 to one of the internal boxes. You use reverse proxy so that DNS calls for Rumpus, FileMaker, Kerio, Synology, munki and munkireport send port 80 and 443 calls to the correct internal i.p. address.

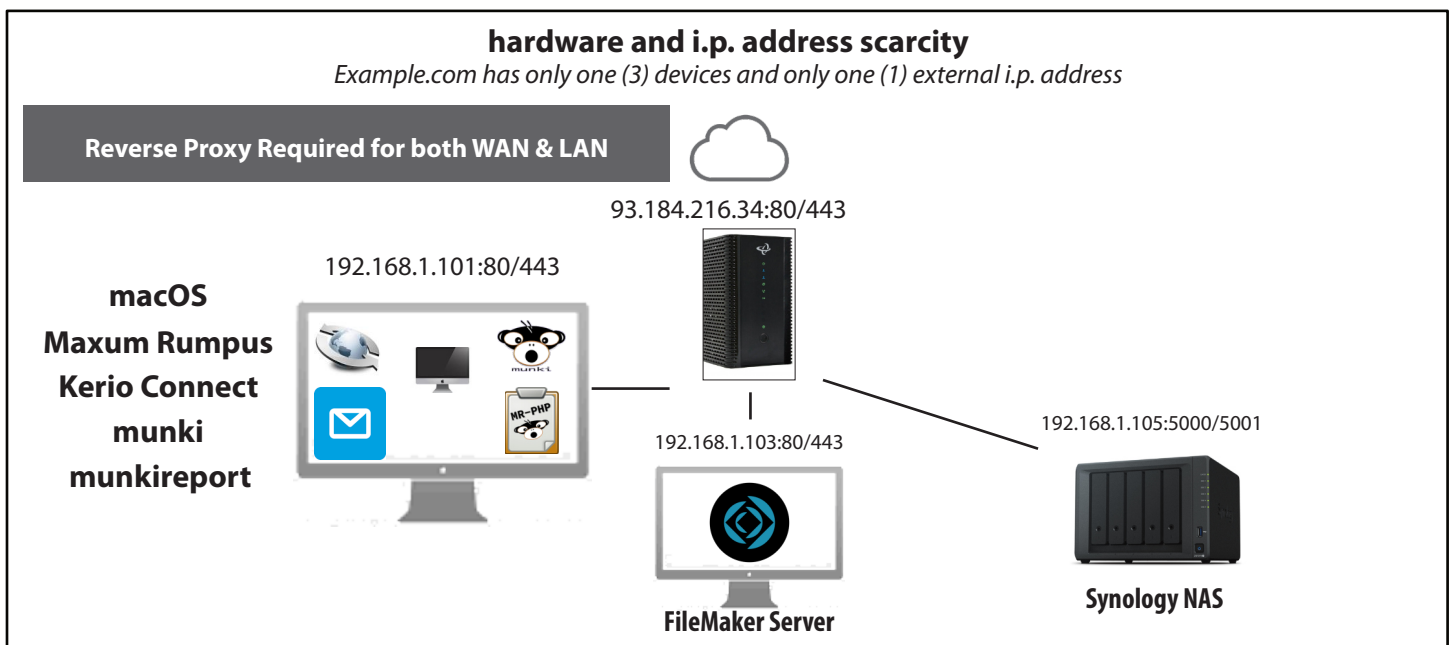


Fig. 3 There is only one external i.p. address and fewer devices than services. The Router passes port 80 and 443 to the server and you use reverse proxy so that DNS calls for Rumpus, FileMaker, Kerio, Synology, munki and munkireport send port 80 and 443 calls to the correct service running on altered ports.

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

First lets get our services, dns and ports sorted out (fig. 4):

Server	FQDN	HTTP	HTTPS
macOS	macOS.example.com	80	443
Maxum Rumpus	rumpus.example.com	8000	8010
FileMaker Server	filemaker.example.com	80	443
Kerio Connect	kerio.example.com	8003	8013
Synology	synology.example.com	5000	5001
munki	munki.example.com	80	443
munkireport	munkireport.example.com	80	443
homebridge	home.example.com	80	

Fig 4. DNS and port assumptions

Overview

1. Set up DNS.
2. Run the setup script: `reverseproxy_setup.sh`
3. Populate certificates folder: `/etc/apache2/certificates`
4. Edit the sites files as necessary in the new folder: `/etc/apache2/sites`
5. uncomment the appropriate include instances in: `/etc/apache2/extra/httpd-vhosts.conf`
6. Test apache with commands...
7. Restart apache using the included `apache_restart.command` in the *Apache commands* folder
8. Make a backup using the included `reverseproxy_backup.command`

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Detailed Instructions

1. Set up DNS.

I am going to assume you know what you are doing here as it is beyond the scope of this tutorial to deal with this. You should have an A record for the Fully Qualified Domain Name (FQDN) of your server:

e.g. apache.example.com

And you should have aliases (CNAMEs) for things like Rumpus, Filemaker Server, Kerio Connect , munki, and munkireport (as per figure 4 above).

2. Run the setup script to enable the appropriate modules and set up the necessary folders and config files

`reverseproxy_setup.sh`

3. Populate the new certificates folder

If you are using SSL get the certificates (not covered here) and place them in the new certificate directory and make sure to set the proper permissions:

/etc/apache2/certificates

4. Edit the sites files as necessary in the sites folder

/etc/apache2/sites

Paycareful attention to the accompanying guide: *Reverse_Proxy_Examples_Guide*

5. uncomment the appropriate *include* instances in:

/etc/apache2/extra/httpd-vhosts.conf

Again, paying careful attention to the accompanying guide: *Reverse_Proxy_Examples_Guide*

6. Test Apache with commands in the included *Apache commands* folder :

`apache_configTest.command`

`apache_fullTest.command`

And correct any errors you may find.

7. Restart apache

`apache_restart.command`

from the included *Apache commands* folder

7. Make a backup

`reverseproxy_backup.command`

from the included *Apache commands* folder

This will back up the **sites**, **certificates**, **extras**, and **conf** modifications we made in case Apple updates apache in a new macOS upgrade and wipes that info (if that happens you can just run the included *reverseproxy_restore.command* after the macOS upgrade to bring everything

Its assumed that you know how to set up munki and munkireport but we do have some tips on Rumpus, Kerio Connect and FileMaker Server below...

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Rumpus

Lets start with Maxum Rumpus as an example since it defaults to port 8000 for web. Download Rumpus from www.maxum.com and place it in your Applications folder. Launch Rumpus and in its SetUp Assistant step 3 Enable the WFM for port 8000:

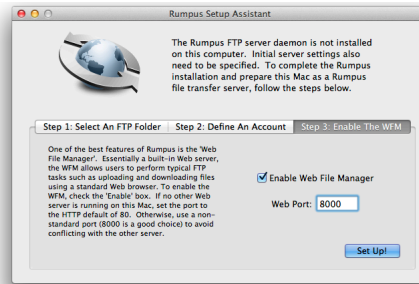


Fig 5. Enable WFM on port 8000 in Rumpus

You may also want to enable secure web services for Rumpus. This is done in the Network Settings: Secure Services tab. For this example install we chose port 8010 as the HTTPS port.

You will also want to tell Rumpus that it is going to be running behind a reverse proxy. Do so by choosing "Open Config Folder" from the "File" menu. Then modify the "Rumpus.conf" file to include the line:

WFMThruProxy YES

If you are also going to be using Rumpus with SSL then you will need to add this to the "Rumpus.conf" file:

HTTPSThruProxy YES

Then save the file, and stop and restart Rumpus.

We assume you can deal with certificates here either self-signed or commercial – In Appendix A you will some info about moving certificates between OS X and these apps.

FileMaker Server

That was easy. Now let's do FileMaker Server. We no longer support FileMaker Server running on anything but its own dedicated device so it can use the normal ports 80 and 443.

Kerio Connect

Download Kerio Connect and the Kerio Connect Open Directory Plugin.

We will reset Kerio HTTP and HTTPS services to use ports 8003 and 8013 respectively.

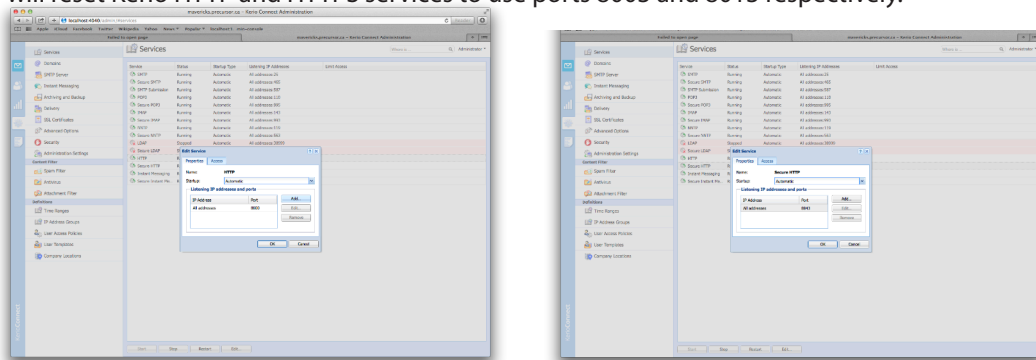


Fig 6 a & b. Turning off default pots of 80, 8800 and 443, 8843 in Kerio Connect

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Incidentally, you will also want to edit the ports for LDAP and Secure LDAP from there default values. This will avoid error messages as the Kerio services cannot run on the same ports as the OS X Server LDAP which was activated earlier when we turned on Open Directory. For this we can just add 99 to the end of the number.

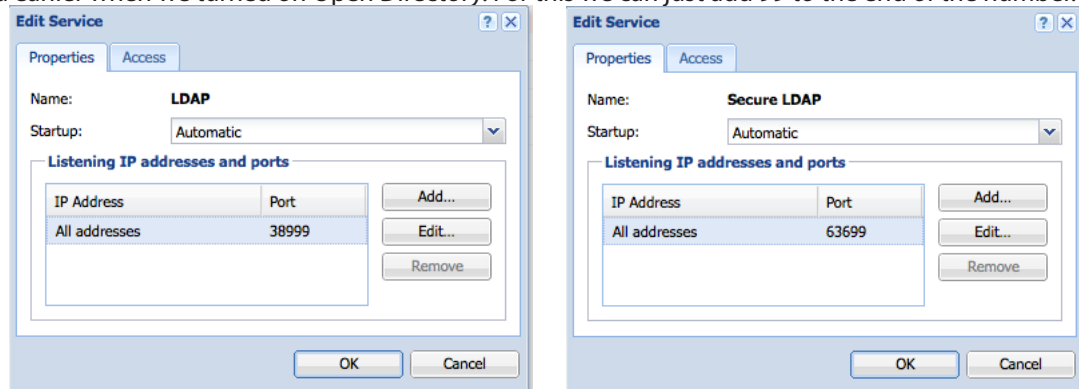


Fig 7 a & b. Editing the LDAP and Secure LDAP ports in Kerio Connect

Permissions

The permissions for things like certificate files need to be owner: root and group: wheel, 644.

You can easily apply this in Terminal:

```
sudo chown -R root:wheel /etc/apache2/certificate/my_certificate.key
```

```
sudo chmod -R 644 //etc/apache2/certificate/my_certificate.key
```

but I like to use the GUI of the utility Batchmod which is available at:
<http://www.lagentesoft.com/Batchmod/>

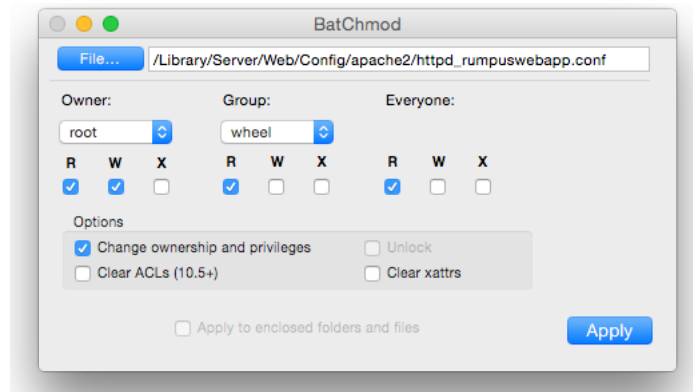


Fig 8. Using BatchMod to modify permissions

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
 support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Revision History:

- 3.17 Oct/1/26. Tested on Tahoe and Golden Gate Updates to docs to mention Golden Gate. Added Appendix C for notes on Google Chrome balking on self-signed certs.
- 3.16 Oct/24/24. Tested on Sequoia. Updates to docs to mention Sequoia. Correct Site File permissions after deployment. Add home-bridge.conf for Synology homebridge. Add Paul Royse methods for FileMaker Websockets and Azure Oauth to FileMaker.conf
- 3.15 Jan/30/22. Fix permissions on Site files. Add better rewrite rule for https. Add ProxyRemote directive for same computer site files
- 3.14 Oct/3/20. Update ProxyTimeout for Kerio to avoid ActiveSync errors. Rewrite SSL Certificate instructions (Appendix A).
- 3.13 Sept/28/20. Numerous typo fixes, update SSL config for TLS 1.2. Some permission fixes for editing purposes.
- 3.12 Aug 10/20. Numerous fixes, code-signed commands, added a set certificate permissions command. Improved documentation.
- 3.1.0 May 6/20. Provided scripting for backup and restore plus a full slate of example host files and include statements to match.
- 3.0.0 March 23/20. FMReverse Proxy using the native apache in macOS 10.14 Mojave and 10.15 Catalina.
- 2.0.6 March 17/20. Fixed two example files..
- 2.0.5 March 15/20. Added a missing example file.
- 2.0.4 March 14/20. Added information on Reverse Proxy to Synology NAS and fixed various typos in documentation and example files.
- 2.0.3 October 1/17. Tested on macOS High Sierra 10.13/macOS Server 5.4. Added some examples for the one external i.p./ multiple internal computers case. FileMaker Server 16.0.2 WebDirect is now working with reverse proxy!
- 2.0.2 September 24/16. Re-edited for macOS Sierra and macOS Server 5.2. Included "SSLProxyEngine on" for SSL conf files necessitated by changes in macOS Server 5.2. Fixed a problem in FixKerioProxyTimeout.sh for macOS Server 5.2. Added Appendix F: Troubleshooting.
- NB. FileMaker routines do not work - waiting for Sierra compatible FileMaker Server to test again.**
- 2.0.1 August 5/16. Amended FileMaker web app conf for FileMaker Server 15 compatibility. Updated throughout to refer to Server 5.1.7.
- 2.0.0 June 12/16. Completely revised for El Capitan and Server app 5.1.5.
Includes critical info for dealing with the Kerio Connect proxy timeout issue.
- 1.0.10 October 24/15. Corrected for OS X Server 5.0.15 fixes and revised instructions for v.3.x and v.4.x to be inline with v.5.0.15.
- 1.0.9 October 10/15. Corrected typos in Appendix C: Server 5 and in the kerio example files.
- 1.0.8 October 1/15. Revised with a working solution for the recently Server app 5.
Changed ports for Kerio to 8003 and 8013 to avoid conflicts with Server 5,
Renamed example files from "fms13" to "filemaker" for clarity.
Detailed Info on OS X Server 5 appears in newly added Appendix C.
- 1.0.7 July 14/15. Added an Appendix B to cover Reverse Proxy to a second internal server
- 1.0.6 May 31/15. Now includes Instructions for FileMaker Server 14.
- 1.0.5 May 11/15. Corrections in the certificates appendix and corrections of typos in the example files.
- 1.0.4 January 24/15, Important config info about using commercial certs (GoDaddy etc); Fixed typo in httpd_Keriowebapp.conf example file;.
- 1.0.3 December 30/14, Re-edited for Yosemite and Server.app 4. Added file ownership and permission info.
- 1.0.2 September 29/14, Added Appendix A with info on importing/Exporting SSL certificates.
- 1.0.1 July 22/14, Add reference to Automatica's AppleScript
- 1.0 July 19/14, Show how to set up reverse proxies for Rumpus, Kerio Connect, and FileMaker Server 13 on OS Mavericks with Server.app 3.

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix A: Working with SSL certificates

It should be obvious that if you are reverse proxying through Apple's implementation of apache, the service you use behind that apache must be using the same SSL Security Certificate as the the one you use in apache.

macOS Server had SSL management capabilities that dovetailed with Keychain Access and previous versions of this tutorial provided instructions for importing to and exporting from the keychain using .p12 files.

However, with native apache we are just manually placing the certificate private and public keys, and intermediate bundle certs in the folder:

/private/etc/apache2/certificates

So it is relatively easy to just use OpenSSL command line calls to create the .csr, .crt, and .key files.

While macOS Catalina and higher no longer use OpenSSL – they use LibreSSL – fortunately the same commands apply.

Creating a .csr (certficate Signing Request) and private .key file.

Open Terminal and type:

```
openssl req -new -newkey rsa:2048 -nodes -keyout example.key -out example.csr
```

eg. (where example_com reflects your own domain)

```
openssl req -new -newkey rsa:2048 -nodes -keyout /Users/Shared/example_com.key -out /Users/Shared/example_com.csr
```

You will be asked to enter the following information that will be incorporated into your certificate request. Note: Ensure to use only alphanumeric characters when filling in the details.

Country Name the two-letter code of your country

State or Province Name the complete name of your state or province

Locality Name your city

Organization Name the officially registered "legal" name for your business.

Organization Unit Name a division or department or the "doing business as" name;

Common Name the FQDN (fully qualified domain name) (ex.: domain.com or sub.domain.com)

Email Address (NB. the e-mail used for the CSR generation will not be used validation or reception of the cert).

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Challenge Password you can leave this field blank.

Optional Company Name can also be left blank.

Once you have generated the .csr and .key files you can open the .csr in a text editor and copy and paste everything between:

--BEGIN CERTIFICATE REQUEST--

and footer:

--END CERTIFICATE REQUEST--

into the Certificate Signing Request application at your SSL vendor.

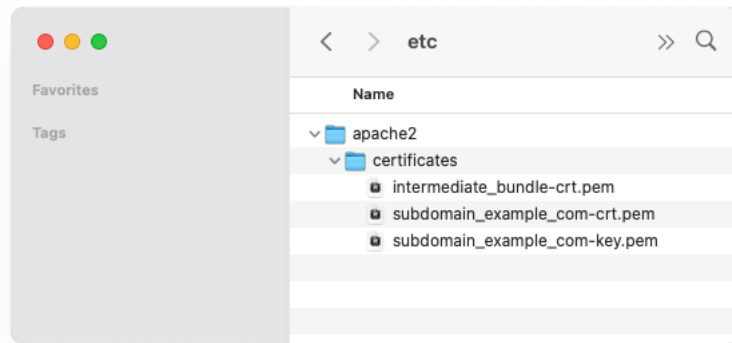


Fig 9. installed certificates for "subdomain.example.com"

Once the certificates are placed in the folder they should have the corrected permissions assigned and you can use the:

`reverseproxy_set_cert_permissions.command`

NB. More info about importing and exporting certificates from FileMaker, Kerio Connect, Maxum Rumpus and Keychain Access can be found in earlier versions of this tutorial.

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix B: TLS 1.2

The default SSL Cipher Suite and Protocol instructions in Apple's example files were only getting a 'B' rating at ssllabs.com tester but fortunately Apple provided instructions in the comments that gives the more secure settings. Therefore, this version of the example files incorporates these changes - which should also apply to `http-ssl.conf`

These lines should be commented out now (with a hash "#"):

```
#SSLCipherSuite HIGH:MEDIUM:!MD5:!RC4:!3DES
```

```
#SSLProxyCipherSuite HIGH:MEDIUM:!MD5:!RC4:!3DES
```

These lines should now be uncommented (remove the hash "#"):

```
SSLCipherSuite HIGH:MEDIUM:!SSLv3:!kRSA
```

```
SSLProxyCipherSuite HIGH:MEDIUM:!SSLv3:!kRSA
```

These lines should be changed from:

```
SSLProtocol all -SSLv3
```

```
SSLProxyProtocol all -SSLv3
```

To:

```
SSLProtocol -all +TLSv1.2
```

```
SSLProxyProtocol -all +TLSv1.2
```

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880

Appendix C: Google Chrome and Self-Signed Certs

When Google Chrome passes through a reverse proxy it detects the primary certificate along with the target certificate. Errors will result if the primary certificate is self-signed.

ie. Lets say the primary A record for the server is *macos.example.com*. Previously, that certificate could be a self-signed certificate while only the certificate for something reverse proxied like *munki.example.com* would have to be a commercial obtained SSL or LetsEncrypt SSL.

But now, Google Chrome will detect that self-signed certificate of *macos.example.com* even though the call is for *munki.example.com* and Chrome will throw errors and not work properly.

Precursor Systems

An Apple Technical Partner

386 Cordova Street • Winnipeg, MB • Canada, R3N 1A6
support@precursor.ca • <http://www.precursor.ca> • (204) 793-1880